Originally, data had to be in cloud object storage before being utilized on the DI Platform. Now, **Query Federation** allows customers to query their data no matter where it resides (see the documentation for any possible limitations) without first worrying about ingestion and data engineering from that remote system. However, that is data in memory, not persistent data. You can land your data in cloud storage in various ways. There are many documentation sources and external tools for the actual landing of data in cloud storage. These may depend on your cloud service provider of choice. Despite best practices of storing data in your cloud storage, using the **Databricks File System** (**DBFS**) to store the data for this book's example projects is also possible.

DBFS is a shared filesystem provided by Databricks that all users of a given workspace can access. Any data stored in DBFS is potentially accessible to all users, regardless of their group, role, or permissions. Therefore, only non-sensitive and non-production data you are willing to share openly across your organization should be in DBFS. An example of non-sensitive data would be the publicly available *Kaggle* datasets. This lack of governance is why we recommend storing data in Databricks volumes, where you can apply governance. We will cover more on volumes in the last section of this chapter.

When it comes to cloud storage, the optimal format for structured data is almost always Delta, which we talked about in detail in *Chapter 1*. When a table is stored in the Delta format, we refer to it as a Delta table. You can choose tables to be "managed" or "external" tables. We use both types of tables in this book (the choice is justified when required). Please see the resources in *Further reading* for more information on the two types of tables.

Discussing source control

Whether or not to use a source control is usually not the question. The question is how should someone use it? Databricks has a few features that can aid in source control.

The first is the version history that is built into notebooks. The history of changes for each notebook is tracked even before submitting it to a remote repository using Git. Version tracking is beneficial as we are not always ready to make a Git commit but still want to track progress and collaborate. It's also a game changer if you accidentally pull someone's code into your working remote branch and forget to push your code before it. The notebook's history will keep your edited copy so you can simply roll back in time and restore all your work!

The second feature is the ease of connecting notebooks and files in your workspace to a remote Git repository. Historically, saving Jupyter notebooks to remote repositories was a technical nightmare for code reviews, sharing, and diffs. The Databricks code repository integration allows Databricks notebooks to contain multiple languages (Python, Scala, SQL) and track them as nicely as a typical Python file. This ability to track notebooks in source as a standard file is an improvement for data engineers and scientists wanting to review notebooks compared to previously converting files to Python and losing all output and images. The days of setting up hooks to automatically save your notebook as a regular Python file every time you save your notebook are over.